

HustleApp Web Frontend Correction Requirements

Document purpose

This document defines the corrections required on the HustleApp web frontend based on the current API behaviour and production database structure.

The frontend developer is expected to correct:

- Data presentation.
- API consumption.
- State management.
- Authentication feedback.
- Financial display.
- Profile persistence on the interface.

The developer must not redesign the backend, introduce new database fields, or calculate financial values independently of the API.

1. Provider Marketplace Card Correction

Current problem

The marketplace currently presents one service as the artisan's main identity.

Example:

Mitchel Bechei – Nails

This is misleading when the provider also offers:

Hairstyling
Dressmaking
Makeup
Other services

The service title must not be displayed as though it is the provider's permanent professional title.

Available API information

The provider-discovery response already contains information such as:

```
artisan_name  
artisan_bio  
category_names  
services_count  
primary_service  
services_endpoint
```

`primary_service` is only one representative service. It is not the provider's complete identity.

Required frontend layout

The provider card should follow this structure:

```
Provider name  
  
Service categories  
  
Number of available services  
  
Featured service, where appropriate  
  
Rating  
  
Location  
  
View Profile button
```

Example:

```
Mitchel Bechei  
  
Beauty, Hairstyling and Fashion  
  
3 services available  
  
Featured service: Nails  
  
★ 4.8 · Accra
```

Incorrect implementation

Do not use:

```
`${provider.artisan_name} – ${provider.primary_service.title}`
```

as the main profile heading.

Recommended implementation

```
function ProviderCard({ provider }) {
  const categories =
    Array.isArray(provider.category_names)
      ? provider.category_names.join(", ")
      : provider.category_names || "Professional services";

  return (
    <article className="provider-card">
      <h3>{provider.artisan_name}</h3>

      <p className="provider-categories">
        {categories}
      </p>

      <p>
        {provider.services_count || 0} services available
      </p>

      {provider.primary_service?.title && (
        <p>
          Featured service: {provider.primary_service.title}
        </p>
      )}

      <button type="button">
        View Profile
      </button>
    </article>
  );
}
```

Acceptance criteria

The correction is complete when:

1. The provider's name is the primary heading.
 2. A service title is no longer presented as the provider's profession.
 3. Multiple categories are displayed where available.
 4. The number of available services is displayed.
 5. The featured service is visually secondary.
 6. Clicking "View Profile" still opens the complete provider profile.
-

2. Provider Proposal Payout Correction

Current problem

When a provider submits a proposal of:

GHS 2,400

the frontend displays:

You'll receive GHS 2,280

This means the frontend is deducting a fee locally.

Under the current backend arrangement, the client bears the platform fee. The provider's displayed estimated payout should not be reduced by a frontend calculation.

Required API fields

The frontend should use:

```
offered_amount
provider_net_estimate
platform_service_fee_amount
service_fee_charged_to
```

Correct provider display

Proposal amount: GHS 2,400

Estimated payout: GHS 2,400

Where appropriate, the interface may also display:

The platform charge is payable by the client.

Prohibited implementation

Remove calculations such as:

```
const providerPayout = proposalAmount * 0.95;
```

or:

```
const providerPayout =  
  proposalAmount - platformServiceFee;
```

Correct implementation

```
const proposalAmount =  
  Number(application.offered_amount || 0);  
  
const estimatedPayout =  
  Number(  
    application.provider_net_estimate ??  
    application.offered_amount ??  
    0  
  );
```

Render:

```
<p>  
  Proposal amount: GHS {proposalAmount.toFixed(2)}  
</p>
```

```
<p>  
  Estimated payout: GHS {estimatedPayout.toFixed(2)}  
</p>
```

Important rule

The frontend must not assume that the platform fee is always:

5%

The fee value must come from the API.

Acceptance criteria

1. A GHS 2,400 proposal displays a GHS 2,400 estimated payout where returned by the API.
2. No percentage is deducted locally.
3. `provider_net_estimate` is treated as the source of truth.
4. The client-side charge is not shown as a provider deduction.

3. Client Invoice and Checkout Breakdown

Current problem

Clients see a final total without enough explanation, causing the total to appear inflated.

The frontend must display every financial component separately.

Required API fields

Use the financial values returned by the API:

```
provider_quote_amount  
platform_service_fee_amount  
insurance_amount  
provider_net_amount  
total_amount_due  
currency
```

The frontend must not calculate the final total using assumed percentages.

Required invoice layout

Example:

```
Service amount:      GHS 400.00
Platform service fee: GHS 60.00
Insurance premium:   GHS 80.00
-----
Total payable:       GHS 540.00
```

Required implementation pattern

```
const breakdown = {
  serviceAmount: Number(job.provider_quote_amount || 0),
  platformFee: Number(
    job.platform_service_fee_amount || 0
  ),
  insurance: Number(job.insurance_amount || 0),
  total: Number(job.total_amount_due || 0),
  currency: job.currency || "GHS",
};
```

Render each line separately.

Prohibited implementation

Do not use:

```
total = amount + amount * 0.05 + amount * 0.15;
```

Do not hard-code:

```
5%
15%
20%
```

inside the checkout interface.

Error handling

When the API does not return a valid breakdown:

```
if (!Number.isFinite(breakdown.total)) {  
  showError(  
    "The payment breakdown could not be loaded. Please try again."  
  );  
  return;  
}
```

Do not generate an estimated total on the frontend.

Acceptance criteria

1. Every charge has a clear label.
2. The total matches `total_amount_due`.
3. No fee percentage is hard-coded.
4. The currency returned by the API is displayed.
5. The provider amount, platform fee and insurance are not merged under one unclear "service charge" label.

4. Google Login and Signup Separation

Current problem

The login and signup buttons previously used the same Google authentication flow without telling the backend whether the user intended to log in or create an account.

The backend has now been updated to support:

```
intent=login  
intent=signup
```

The frontend must send the appropriate intent.

4.1 Google Login

The login page must request:

```
GET https://api-v2.hustleapp.info/api/v1/auth/google/url  
?intent=login
```

&platform=web

&return_to=https://v2.hustleapp.info/sign-in

Example:

```
async function startGoogleLogin() {
  const params = new URLSearchParams({
    intent: "login",
    platform: "web",
    return_to: "https://v2.hustleapp.info/sign-in",
  });

  const response = await fetch(
    `https://api-v2.hustleapp.info/api/v1/auth/google/url?${params}`
  );

  const result = await response.json();

  if (
    !response.ok ||
    result.success !== true ||
    !result.data?.auth_url
  ) {
    throw new Error(
      result.message || "Unable to start Google login."
    );
  }

  window.location.assign(result.data.auth_url);
}
```

Expected unregistered-user response

The frontend must handle:

GOOGLE_ACCOUNT_NOT_REGISTERED

Display:

No HustleApp account was found for this Google email.
Please sign up first.

Provide a visible action:

Create an account

The login flow must not create a new account.

4.2 Google Signup

The signup page must send:

```
intent=signup
account_type
country_iso2 or country_id
timezone_name
platform=web
return_to
```

Example request:

```
GET https://api-v2.hustleapp.info/api/v1/auth/google/url
?intent=signup
&account_type=client
&country_iso2=GH
&timezone_name=Africa/Accra
&platform=web
&return_to=https://v2.hustleapp.info/sign-up
```

Example implementation:

```
async function startGoogleSignup({
  accountType,
  countryIso2,
}) {
  if (!accountType) {
    throw new Error(
      "Please select an account type."
    );
  }

  if (!countryIso2) {
    throw new Error(
      "Please select your country."
    );
  }
}
```

```

const timezoneName =
  Intl.DateTimeFormat()
    .resolvedOptions()
    .timeZone || "Africa/Accra";

const params = new URLSearchParams({
  intent: "signup",
  account_type: accountType,
  country_iso2: countryIso2,
  timezone_name: timezoneName,
  platform: "web",
  return_to: "https://v2.hustleapp.info/sign-up",
});

const response = await fetch(
  `https://api-v2.hustleapp.info/api/v1/auth/google/url?${params}`
);

const result = await response.json();

if (
  !response.ok ||
  result.success !== true ||
  !result.data?.auth_url
) {
  throw new Error(
    result.message || "Unable to start Google signup."
  );
}

window.location.assign(result.data.auth_url);
}

```

Expected existing-account response

Handle:

GOOGLE_ACCOUNT_ALREADY_EXISTS

Display:

An account already exists for this Google email.
Please sign in instead.

Provide:

Go to login

Google authentication success rule

The frontend must not consider HTTP status `200` alone as successful authentication.

Authentication is successful only when:

```
function isValidAuthToken(result) {  
  return (  
    result?.success === true &&  
    typeof result?.data?.access_token === "string" &&  
    result.data.access_token.trim().length > 0  
  );  
}
```

Do not store authentication state when the token is missing.

Error codes to support

```
GOOGLE_ACCOUNT_NOT_REGISTERED  
GOOGLE_ACCOUNT_ALREADY_EXISTS  
GOOGLE_ACCOUNT_TYPE_REQUIRED  
ACCOUNT_TYPE_REQUIRED  
COUNTRY_REQUIRED  
INVALID_GOOGLE_AUTH_INTENT  
GOOGLE_AUTHENTICATION_FAILED
```

Acceptance criteria

1. Login sends `intent=login`.
2. Signup sends `intent=signup`.
3. Signup sends account type and country.
4. Unregistered login users receive "Please sign up first."
5. Existing signup users receive "Please sign in instead."

6. No user is treated as logged in without an access token.
 7. Generic errors are not displayed when the API returns a specific message.
-

5. Profile Data Persistence and State Refresh

Current problem

Date of Birth, Gender and Default City may disappear from the interface after logout and login.

The frontend must not rely only on:

```
Local state  
Cached profile data  
Login response  
Partial update response
```

Required profile fields

The interface must correctly map:

```
date_of_birth  
gender  
default_city_id
```

It must also preserve:

```
first_name  
middle_name  
last_name  
phone_number  
bio  
profile_image_asset_id
```

Required load behaviour

After successful login:

```
GET /api/v1/profile
```

After a successful profile update:

```
GET /api/v1/profile
```

The returned profile must replace the current profile state.

Recommended implementation

```
async function loadProfile(token) {
  const response = await fetch(
    "https://api-v2.hustleapp.info/api/v1/profile",
    {
      headers: {
        Authorization: `Bearer ${token}`,
        Accept: "application/json",
      },
    }
  );

  const result = await response.json();

  if (!response.ok || result.success !== true) {
    throw new Error(
      result.message || "Unable to load profile."
    );
  }

  return result.data;
}
```

After update:

```
async function saveAndRefreshProfile(
  token,
  profilePayload
) {
  const updateResponse = await fetch(
    "https://api-v2.hustleapp.info/api/v1/profile",
    {
      method: "PATCH",
      headers: {
        Authorization: `Bearer ${token}`,
        "Content-Type": "application/json",
        Accept: "application/json",
      },
    }
  );
}
```

```

    },
    body: JSON.stringify(profilePayload),
  }
);

const updateResult = await updateResponse.json();

if (
  !updateResponse.ok ||
  updateResult.success !== true
) {
  throw new Error(
    updateResult.message ||
    "Unable to update profile."
  );
}

return await loadProfile(token);
}

```

Form mapping requirements

Date of birth

```

form.dateOfBirth =
  profile.date_of_birth || "";

```

Gender

```

form.gender =
  profile.gender || "";

```

Default city

```

form.defaultCityId =
  profile.default_city_id
    ? String(profile.default_city_id)
    : "";

```

Important rules

1. Do not replace a valid API value with a placeholder.
2. Do not treat an empty local cache as proof that the server has no value.

3. Fetch the profile after every login.
4. Fetch the profile after every successful update.
5. Do not store only the profile-update response if it is incomplete.
6. Clear profile state completely on logout before another user logs in.

Acceptance criteria

1. Enter Date of Birth, Gender and Default City.
2. Save the profile.
3. Refresh the page.
4. Confirm the values remain.
5. Log out.
6. Log in again.
7. Confirm the values still appear.
8. Update another unrelated field, such as phone number.
9. Confirm Date of Birth, Gender and Default City remain visible.

6. Hustle Listing on the Web Provider Dashboard

Where the web frontend includes a provider hustle feed, it must use:

```
GET /api/v1/hustles?status=open&page=1&per_page=20
```

It must not use:

```
GET /api/v1/my/hustles
```

for provider discovery.

`/my/hustles` is intended for clients and companies to see posts they created.

Initial test request

Use:

```
GET https://api-v2.hustleapp.info/api/v1/hustles
?status=open
&page=1
&per_page=20
```

Do not initially send:

```
country_id
city_id
category_id
date_from
preferred_date
```

unless the user selected those filters.

Required diagnostics during development

Log in development mode:

```
Request URL
Query parameters
HTTP status
Number of API items
Number of rendered cards
```

Do not expose access tokens in console logs.

Acceptance criteria

1. Open client-created hustles returned by the API are displayed.
2. Company-created hustles returned by the API are displayed.
3. No hidden local filter removes valid posts.
4. Empty-state text appears only when the API returns no matching items.
5. Pagination loads additional items correctly.
6. The feed refreshes after returning to the page.

7. Loading, Error and Empty States

Every affected interface must have three distinct states.

Loading

```
Loading providers...
Loading profile...
Loading available hustles...
Preparing Google authentication...
```

Empty result

```
No providers are available for the selected filters.  
No open hustles were found.  
No services have been added by this provider.
```

Error

Display the API message where available:

```
const errorMessage =  
  result?.message ||  
  "Something went wrong. Please try again.";
```

Do not use one generic message for all authentication, payment and profile errors.

8. Frontend Data Rules

The frontend must follow these general rules:

1. The API is the source of truth for financial values.
 2. Do not hard-code platform-fee percentages.
 3. Do not hard-code insurance percentages.
 4. Do not deduct client charges from the provider's payout.
 5. Do not represent one service as the provider's complete profession.
 6. Do not assume HTTP 200 means authentication succeeded.
 7. Do not save authentication without an access token.
 8. Do not depend only on cached profile data.
 9. Do not send filters that the user did not select.
 10. Do not expose access tokens in logs.
 11. Handle expired sessions by returning the user to login.
 12. Display backend validation messages clearly.
-

9. Items Outside the Frontend Scope

The frontend developer should not implement the following as part of this correction:

```
Mobile Money withdrawal integration  
Database schema changes
```

New professional-title field
New certificate file formats
Push queue-worker implementation
OTP email delivery changes
Platform-fee policy changes
Insurance-rate policy changes

These are backend, mobile, product or new-feature decisions.

The frontend should only display the financial configuration returned by the API.

10. Required Frontend Delivery Evidence

The frontend developer must submit:

1. Updated source code.
2. Build file or deployment package.
3. Screenshots of the corrected provider cards.
4. Screenshot of a GHS 2,400 provider proposal showing the correct payout.
5. Screenshot of the complete client invoice breakdown.
6. Video of registered Google-user login.
7. Video of unregistered Google-user login showing "Sign up first."
8. Video of new Google signup.
9. Video of profile data remaining after logout and login.
10. Video of open hustles loading on the provider dashboard, where applicable.
11. Network screenshots showing the endpoints used.
12. A list of files modified.

Access tokens, passwords and private user information must be hidden in screenshots and recordings.

11. Frontend Completion Checklist

The frontend task will be considered complete when:

- ☐ Provider cards represent the artisan, not one project.
- ☐ Multiple service categories are displayed.
- ☐ Service count is displayed.
- ☐ Provider proposal payout is not reduced locally.
- ☐ Client invoice uses the API breakdown.
- ☐ Login sends `intent=login`.
- ☐ Signup sends `intent=signup`.
- ☐ Google authentication errors are displayed correctly.
- ☐ The user is not authenticated without a token.

- ☐ Profile data is fetched after login.
- ☐ Profile data is fetched after updates.
- ☐ Date of Birth persists.
- ☐ Gender persists.
- ☐ Default City persists.
- ☐ Open hustles returned by the API are displayed.
- ☐ Loading, empty and error states are implemented.
- ☐ No financial percentage is hard-coded.
- ☐ No new backend or database feature is introduced.